ASIC Design and Automation Project

Project 1: FSM Front-End Design

Student Name	Code	Email	Department
Ahmed Haitham Othman	2100493	AhmedHaithamOthman@gmail.com	ECE Senior 2
Mohamed Adel Abdelrahem	2100395	mhmdadel344@gmail.com	ECE Senior 2
Yahia Ashraf Emam	2101013	yahiaashraf2014@gmail.com	ECE Senior 2

content	page

Chapter 1	2-4
Chapter 2	4-5
Chapter 3	6-14
Chapter 4	14-19
Chapter 5	19-22

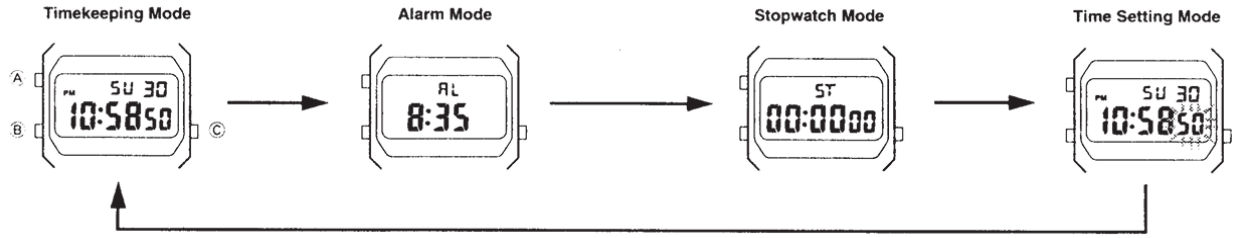
Chapter 1 Introduction

Project idea:

The idea of this project is to design the **front-end digital controller** of a multifunction digital watch using Verilog HDL. The watch must support four essential operating modes—**Normal Time Display, Time Setting, Alarm Setting, and Stopwatch**—while relying on only **two push-buttons** for all user interactions. The design aims to model the behavior of a real embedded watch system by creating a clean hardware architecture composed of multiple RTL blocks and a supervising **Finite State Machine (FSM)**.

Each operational mode is implemented as an independent RTL module. The FSM acts as the top-level controller that selects which block is active, manages transitions between modes, handles digit selection during configuration, and interprets all button inputs. The project demonstrates how a complex user interface can be built from minimal hardware resources by using an organized FSM architecture and modular RTL design. The final system displays four 7-segment digits (hh:mm) and supports alarm triggering and stopwatch functionality.

Example of the Watch (CASIO)



but the watch we will implement have only 2 push buttons (mode,set) not 3 like CASIO (A,B,C) as said in the requirement.

Overall, this project develops core competencies in digital front-end design, FSM modeling, modular RTL architecture, and simulation-based verification — all foundational skills for ASIC development.

Problem description:

Modern digital watch systems must provide multiple features while operating under strict constraints such as limited I/O, low power, and simple user interfaces. In this project, the system is constrained to only **two buttons**:

- **MODE**: used to switch between watch modes and advance through digits
- **SET**: used to increment digits or control stopwatch actions

The challenge is to design a digital system that interprets these limited inputs and still provides all required functionalities:

1. **Normal Mode**
    - Display the current time in 24-hour format (hh:mm)
    - Internal counters must handle hours and minutes with proper rollover
2. **Time Setting Mode**
    - User modifies the four time digits (hour tens → hour units → minute tens → minute units)
    - Only the selected digit is incremented when SET is pressed
3. **Alarm Setting Mode**
    - Similar digit-by-digit configuration for hh:mm alarm time
    - Alarm must trigger when the stored alarm time matches the current time
4. **Stopwatch Mode**
    - Displays mm:ss
    - SET button must control start, stop, and split operations

The main difficulty arises from coordinating these functions using two buttons only. The design must ensure:

- Clear navigation between modes
- Reliable digit selection
- Debounced button inputs
- No conflicts between RTL blocks
- Smooth transitions across all states

To solve this problem, the system is divided into **four RTL functional blocks**, each responsible for a specific operational mode. Above them, a dedicated FSM is implemented to enable exactly one block at a time, interpret button presses, and

generate the control signals required for digit incrementing, stopwatch control, and mode switching.

This structured approach provides a scalable and verifiable hardware design that models a realistic embedded watch system suitable for ASIC front-end development.

Chapter 2 FSM Design

Design details “control architecture”:

The digital watch system is divided into four RTL functional blocks, each responsible for one operating mode:

1. **Normal Mode Block** – displays time and handles minute/hour counting
2. **Set Time Block** – allows digit-by-digit time configuration
3. **Set Alarm Block** – allows digit-by-digit alarm configuration
4. **Stopwatch Block** – implements mm:ss counting, start/stop, and split

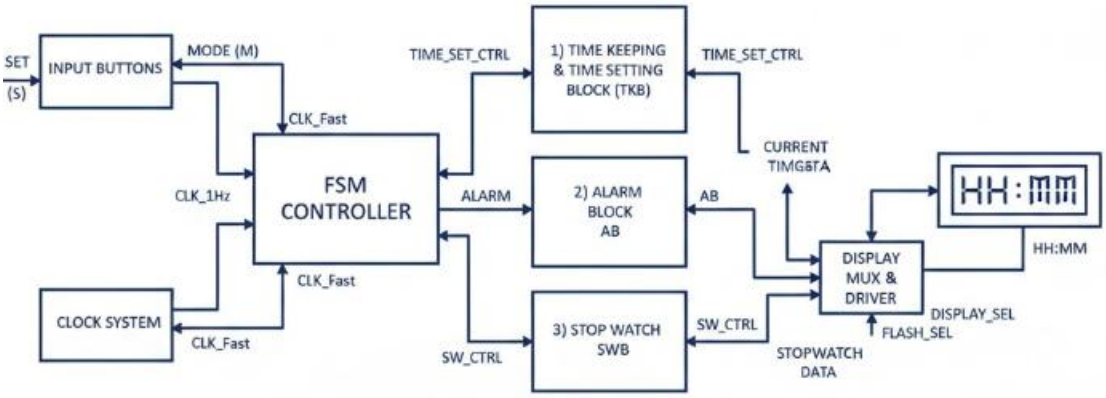
But we made the Normal block mode with set time mode in same BLOCK (it doesn’t matter)

Because the watch uses only **two push-buttons** (**MODE** and **SET**) for all interactions, the system requires a centralized controller that interprets button presses and selects the correct operating mode. This controller is implemented as a **Finite State Machine (FSM)**.

The FSM is responsible for:

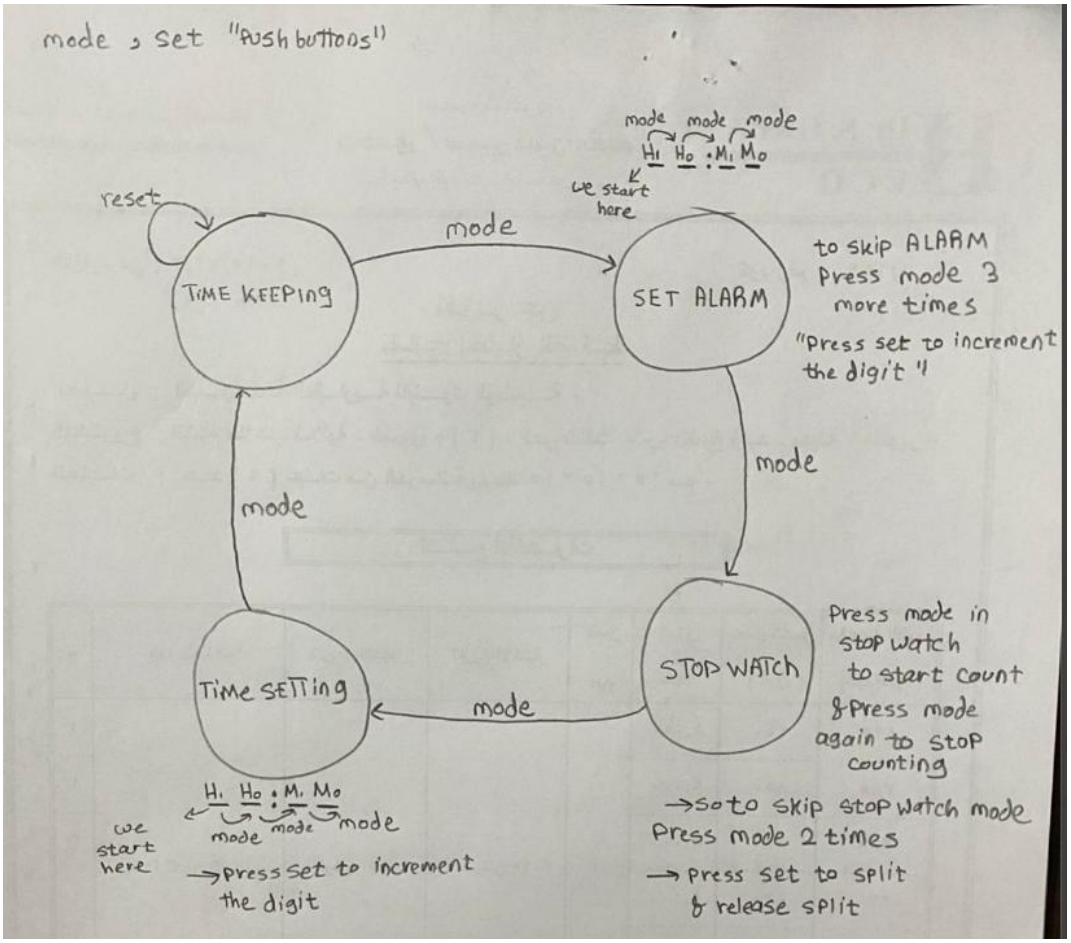
- Enabling one RTL block at a time
- Managing transitions between modes
- Handling digit selection for time/alarm configuration
- Interpreting SET button behavior based on the active mode
- Coordinating stopwatch start, stop, and split operations
- Generating control signals for the Datapath.

System Diagram



FSM Diagram

This is abstract FSM for 4 modes the full FSM will include internal digits.



Chapter 3 RTL design

RTL codes

1st block (time keeping & time setting mode)

Comment: this block used to increment the normal time , or to set the time when the FSM tell the block to do that “enable=1”.

```
module Watch_time (
input wire switch_digit_A,
input wire increment_C,
input wire enable,
input wire CLK,
input wire reset,
output reg done,
output reg [3:0] H1,H0,M1,M0
);

reg [1:0] selected_digit;
reg [5:0] seconds;
always@(posedge CLK or posedge reset)
 begin
    if(reset) begin
    H1 <= 0;
    H0 <= 0;
    M1 <= 0;
    M0 <= 0;
    seconds <= 0;
    selected_digit <= 0;
    end
    else begin
    if(!enable) begin
      selected_digit = 0;
      if(H1 == 2 && H0 == 3 && M1 == 5 && M0 == 9 && seconds == 59) begin
//midnight
      H1 <= 0;
      H0 <= 0;
      M1 <= 0;
      M0 <= 0;
      seconds <= 0;
    end
    else if(H0 == 9 && M1 == 5 && M0 == 9 && seconds == 59) begin //1st hour
digit should increment
```

```
      H1 <= H1 + 1;
      H0 <= 0;
      M1 <= 0;
      M0 <= 0;
      seconds <= 0;
    end
    else if(M1 == 5 && M0 == 9 && seconds == 59) begin //2nd hour digit should
increment
      H0 <= H0 + 1;
      M1 <= 0;
      M0 <= 0;
      seconds <= 0;
    end
    else if(M0 == 9 && seconds == 59) begin //1st Minute digit should increment
      M1 <= M1 + 1;
      M0 <= 0;
      seconds <= 0;
    end
    else if(seconds == 59) begin //2nd Minute digit should increment
      M0 <= M0 + 1;
      seconds <= 0;
    end
    else begin
      seconds <= seconds + 1; //seconds incrementing
    end
    end
    else begin
      if(switch_digit_A == 1)
      selected_digit <= selected_digit + 1;
      if(increment_C == 1) begin
      case (selected_digit)
      2'b00 : if (H1 == 2 || (H1 == 1 && H0 > 3)) //incrementing 1st hour digit
              H1 <= 0;
              else
              H1 = H1 + 1;
      2'b01 : if (H0 == 9 || (H1 == 2 && H0 == 3)) //incrementing 2nd hour digit
              H0 <= 0;
              else
              H0 <= H0 + 1;
      2'b10 : if (M1 == 5) //incrementing 1st minute digit
              M1 <= 0;
              else
              M1 <= M1 + 1;
      2'b11 : if (M0 == 9) //incrementing 2nd minute digit
              M0 <= 0;
```

```
              else
              M0 <= M0 + 1;
      endcase
      end
    end
    end
 end

always@(*) begin
if (selected_digit == 3) //when at the final digit, done flag is high to go to
the next mode
 done = 1;
else
 done = 0;
end
endmodule
```

2nd Block Alarm mode

Comment: this block used to set ALARM to specific time when the FSM put the enable=1.

```
module Alarm (
input wire switch_digit_A,
input wire increment_stop_C,
input wire CLK,//clk in seconds if there is no clk divider
input wire reset,
input wire enable,
input wire [3:0] H1_in,H0_in,M1_in,M0_in,
output reg done,
output reg [3:0] H1,H0,M1,M0,
output reg Alarm_beep
);

reg Alarm_Mode;//Alarm active or not
reg [1:0] selected_digit;
reg [4:0] alarm_count;
always@(posedge CLK or posedge reset)
 begin
    if(reset) begin
    H1 <= 0;
    H0 <= 0;
    M1 <= 0;
    M0 <= 0;
    selected_digit <= 0;
    Alarm_Mode <= 0;
```

```
    alarm_count <= 0;
    Alarm_beep <= 0;
    end
    else if (enable) begin
    if(switch_digit_A == 1)
    selected_digit <= selected_digit + 1;


    if(increment_stop_C) begin
    Alarm_Mode <= 1;//if alarm time is modified then it's active
    case (selected_digit)
    2'b00 : if (H1 == 2 || (H1 == 1 && H0 > 3)) //incrementing 1st hour digit
            H1 <= 0;
            else
            H1 = H1 + 1;
    2'b01 : if (H0 == 9 || (H1 == 2 && H0 == 3)) //incrementing 2nd hour digit
            H0 <= 0;
            else
            H0 <= H0 + 1;
    2'b10 : if (M1 == 5) //incrementing 1st minute digit
            M1 <= 0;
            else
            M1 <= M1 + 1;
    2'b11 : if (M0 == 9) //incrementing 2nd minute digit
            M0 <= 0;
            else
            M0 <= M0 + 1;
endcase
    end
    end
    else
    selected_digit <= 0;
    if (Alarm_beep) begin
    if(alarm_count == 19) begin //after 20 seconds Alarm beep turns off
    Alarm_beep <= 0;
    Alarm_Mode <= 0;
    end
    else
    alarm_count <= alarm_count + 1; //counting how long alarm beep is on
    end
    end

always@(*) begin
    if(H1 == H1_in && H0 == H0_in && M1 == M1_in && M0 == M0_in && Alarm_Mode)
//when the current time matches with the alarm time, alarm_beep is active
```

```
    Alarm_beep = 1;

    if (selected_digit == 3) //when at the final digit, done flag is high to go
to the next mode
    done = 1;
    else
    done = 0;
end

endmodule
```

3rd Block stop watch mode

Comment: we designed the stop watch to count when mode=1 and stop when mode is pressed again , and also we covered the split feature when set=1 , and split release when set is pressed again.

We enter the stopwatch when we press the mode several times to skip the ALARM mode and the digits until reaching the STOP WATCH mode.

```
module stopwatch (
    input   wire        enable, start_C, split_A,
    input   wire        CLK, RST,
    output  wire [3:0]  m1_out, m0_out, s1_out, s0_out,
    output  reg     done
);

reg count;
reg [3:0]   m1_0, m0_0, s1_0, s0_0; //mem 0
reg [3:0]   m1, m0, s1, s0; //original timer

reg mem;    //select of output mux

assign {m1_out, m0_out, s1_out, s0_out} = (!mem) ? {m1, m0, s1, s0} : {m1_0,
m0_0, s1_0, s0_0};

always@(posedge CLK or posedge RST)
    begin
        if (RST)
            begin
                done <= 0;
                count <= 0;
                mem <= 0;
                {m1, m0, s1, s0} <= 0;
                {m1_0, m0_0, s1_0, s0_0} <= 0;
```

```
            end
        else
            begin
                if (enable)
                    begin
                        if (start_C && !count)      count <= 1; //start and stop
stopwatch
                        else if (start_C && count)  begin
                            count <= 0;
                            done <= 1;
                        end

                        if (split_A && !count)
                            begin
                                //clear

                                mem <= 0;
                                {m1, m0, s1, s0} <= 0;
                                {m1_0, m0_0, s1_0, s0_0} <= 0;
                            end
                        else if (split_A && count)  //split
                            begin
                                //store then {display/return to timer}
                                {m1_0, m0_0, s1_0, s0_0} <= {m1, m0, s1, s0};
                                mem <= !mem;
                            end

                        if (count) //stopwatch counter
                            begin
                                if (s0 == 6'd9)
                                    begin
                                        s0 <= 0;
                                        if (s1 == 6'd5)
                                            begin
                                                s1 <= 0;
                                                if (m0 == 6'd9)
                                                    begin
                                                        m0 <= 0;
                                                        if (m1 == 6'd5)
                                                            begin
                                                                m1 <= 0;
                                                            end
                                                        else    m1 <= m1 + 1;
                                                    end
                                                else    m0 <= m0 + 1;
```

```
                                        end
                                    else    s1 <= s1 + 1;
                                end
                            else    s0 <= s0 + 1;
                        end


                    end
                else
                    begin
                        done <= 0;
                        count <= 0;
                        mem <= 0;
                        {m1, m0, s1, s0} <= 0;
                        {m1_0, m0_0, s1_0, s0_0} <= 0;
                    end
            end
    end


endmodule
```

4th Block FSM

Comment: the FSM description is written in the paper showing the FSM diagram, we implemented it here in the code.

```
////////////////////////////////////
///////////// Moore FSM /////////////
////////////////////////////////////

module WATCH_FSM (
input  wire        B, C,
input  wire        reset,
input  wire        CLK,
input  wire        Timesetting_done,Alarm_done,Stopwatch_done,
output reg   Timesetting_enable,Alarm_enable,Stopwatch_enable,
output reg  [1:0]  mux_sel //used to select which mode output is displayed
);

localparam  [1:0]    Timekeeping = 2'b00,
                     Alarm = 2'b01,
                     Stopwatch = 2'b10,
                     Timesetting = 2'b11;

reg    [1:0]         count;
```

```
reg    [2:0]         current_state,
                     next_state ;

// state transition
always @(posedge CLK or posedge reset)
 begin
  if(reset)
   begin
     current_state <= Timekeeping ;
   end
  else
   begin
     current_state <= next_state ;
   end
 end

// next_state logic
always @(*)
 begin
  case(current_state)
  Timekeeping     : begin
              if(B)
               next_state = Alarm ;
              else
               next_state = Timekeeping ;
             end
  Alarm    : begin
               if(B && Alarm_done)
               next_state = Stopwatch ;
              else
               next_state = Alarm ;
             end
  Stopwatch  :  begin
               if(B && Stopwatch_done)
               next_state = Timesetting ;
               else
               next_state = Stopwatch ;
             end
  Timesetting  :  begin
               if(B && Timesetting_done)
               next_state = Timekeeping ;
               else
               next_state = Timesetting ;
             end
  endcase
```

```
end

// next_state logic
always @(*)
 begin
  case(current_state)
  Timekeeping     : begin
              Timesetting_enable  =  1'b0 ;
              Alarm_enable    =  1'b0 ;
              Stopwatch_enable =  1'b0 ;
              mux_sel =  2'b00 ;
              end
  Alarm    : begin
              Timesetting_enable  =  1'b0 ;
              Alarm_enable    =  1'b1 ;
              Stopwatch_enable =  1'b0 ;
              mux_sel =  2'b01 ;
             end
  Stopwatch  :  begin
              Timesetting_enable  =  1'b0 ;
              Alarm_enable    =  1'b0 ;
              Stopwatch_enable =  1'b1 ;
              mux_sel =  2'b10 ;
             end
  Timesetting  :  begin
              Timesetting_enable  =  1'b1 ;
              Alarm_enable    =  1'b0 ;
              Stopwatch_enable =  1'b0 ;
              mux_sel =  2'b00 ;
             end
  endcase
 end

endmodule
```

Chapter 4 TB design

Test scenario

1)<u>test reset so we will start at TIME KEEPING mode</u>

Reset=1, mode=0 , set=0.

2) <u>test SET ALARM to 12:30</u>

Mode=1 , set=1 , mode=1 (to select 2nd digit) , set =1 , set = 1 , mode =1 (to select 3rd digit) , set = 1 , set =1 , set = 1, mode =1 (to select 4th digit) , set =0.

>> wait a delay for observing the alarm beeb

3) <u>test STOP WATCH mode</u>

Mode=1 (to go to STOP WATCH) , mode=1 (to start counting) , mode=0 wait some time to see the counting method , mode=1 (to stop counting).

4) <u>test reset again and TIME SETTING mode to set time to (14:22)</u>

Reset=1 , now we are in TIME KEEPING mode , reset=0 , mode=1 , mode=1 , mode=1 , mode=1 , mode=1 , mode=1 , mode=1 , mode=1 , (8 times to go through ALARM mode and then STOP watch and then TIME SETTING since each click on the mode port it change the current mode and the digit so we press 1st time and go to ALARM and then press 3 times to go through the all digits and then press 1 time to go to STOP WATCH mode and then press 2 times to start and stop counting and last time (8 times) for going to TIME SETTING), set=1 (now we set the first digit to 1) , mode=1 "to select 2nd digit", set=1 , set=1, set=1 , set=1 "now 2nd digit is set to 4" , mode =1 , set =1 , set =1 , mode = 1 , set =1 , set =1. Now the 3rd and 4th digit is set to 2.

5) <u>test split in STOP WATCH</u>

Reset then press mode 5 times to go to stop watch , press mode=1 to start count, press set=1 to split , set=0 wait some time , set=1 "split release" , set=0 , wait some time to continue counting , mode=1 "to stop counting".

TB code

```
// Smart_Watch_TB.v
// reset is active high
// 2 inputs port "mode" and "set" and 4 output port "clock_display" Hour1 , Hour0
, Min1 , Min0
// mode = A,B  ---- set=C;
// mode used for changing between states and select the digit to be set
// set used to increment the selected digit
```

```
`timescale 100ms / 1ns // period = 10 x 100 ms = 1s
module WATCH_TOP_TB (
);

// Signal declaration
reg clk_tb;
reg reset_tb;
reg mode_tb;
reg set_tb;
wire [3:0] Hour1_tb;
wire [3:0] Hour0_tb;
wire [3:0] Min1_tb;
wire [3:0] Min0_tb;
wire Alarm_beep_tb;

// Clock generation
always #5 clk_tb = ~clk_tb;

// Test stimulus
initial begin
    // Initialize inputs
    clk_tb = 0;
    reset_tb = 0;
    mode_tb = 0;
    set_tb = 0;
    #10;
    //1st case: Reset the watch so we will be at state TIME KEEPING mode with
00:00
    reset_tb = 1;
    #10;

    //2nd case: change to SET ALARM mode (12:30)
    reset_tb = 0;
    mode_tb = 1; // B=high
    #10;  // now we are in SET ALARM mode
    mode_tb = 0; // B=low for setting first digit to 1
    set_tb = 1; // C = high to increment
    #10; // increment Hour1 to 1
    mode_tb = 1; // B=high to move to next digit
    set_tb = 0; // C=low
    #10;
    mode_tb = 0; // B=low for setting Hour0
    set_tb = 1; // C = high to increment
    #20; // increment Hour0 to 2
```

```
    mode_tb = 1; // B=high to move to next digit
    set_tb = 0; // C=low
    #10;
    mode_tb = 0; // B=low for setting Min1
    set_tb = 1; // C = high to increment
    #30; // increment Min1 to 3
    mode_tb = 1; // B=high to move to next digit
    set_tb = 0; // C=low
    #10;
    mode_tb = 0; // B=low for setting Min0
    set_tb = 0; // C = low since Min0 we want to set is = 0
    #10; // wait for a moment
    // now the alarm time is set to 12:30

    // make a delay to wait for the time to be 12:30 and check if alarm triggers
    #500000; // wait for a long time to simulate time passing



    //3rd case: change to STOP WATCH mode
    $display("stop watch case starts here %0d%0d:%0d%0d  @simulation time %0d",
Hour1_tb, Hour0_tb, Min1_tb, Min0_tb, $time);
    mode_tb = 1; // B=high to go from last digit in SET ALARM to STOP WATCH mode
    #10;
    mode_tb = 1; // mode=high to START counting
    #10;
    mode_tb = 0; // mode=low to continue in counting
    #100; // wait for some time to let stopwatch count
    mode_tb = 1; // mode=high to stop counting
    #10;


    //4th case: Reset the watch again
    reset_tb = 1;
    mode_tb = 0;
    #10;
    reset_tb = 0;
    #10;
    // now we need to test TIME SETTING mode to (14:22)
    mode_tb = 1; // B=high to go throug modes and digits untill TIME SETTING mode
    #10; #10; #10; #10; #10; #10; #10; #10;  // 8 times to reach TIME SETTING
mode
    mode_tb = 0; // stay in TIME SETTING mode to set Hour1
    set_tb = 1; // C=high to increment Hour1
    #10; // increment Hour1 to 1
```

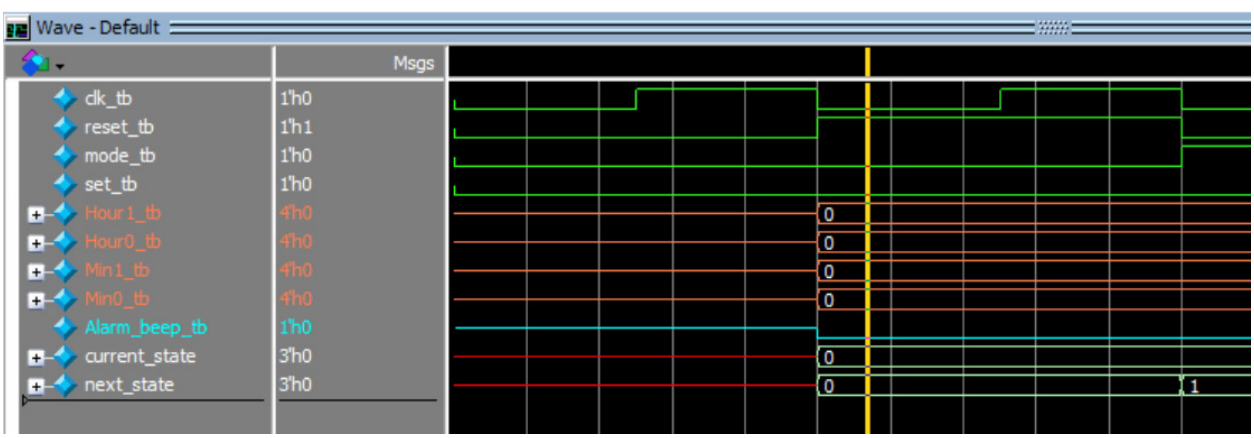
```
    mode_tb = 1; // high to move to next digit Hour0
    set_tb = 0; // C=low
    #10;
    mode_tb = 0; // low to set Hour0
    set_tb = 1; // set digit Hour0
    #40; // increment Hour0 to 4
    mode_tb = 1; // high to move to next digit Min1
    set_tb = 0; // C=low
    #10;
    mode_tb = 0; // low to set Min1
    set_tb = 1; // C=high to increment Min1
    #20; // increment Min1 to 2
    mode_tb = 1; // high to move to next digit Min0
    set_tb = 0; // C=low
    #10;
    mode_tb = 0; // low to set Min0
    set_tb = 1; // C=high to increment Min0
    #20; // increment Min0 to 2
    $display("Time is set to %0d%0d:%0d%0d  @simulation time %0d", Hour1_tb,
Hour0_tb, Min1_tb, Min0_tb, $time);
    // now time is set to 14:22;


    //5th case: test split in STOP WATCH mode
    //first reset to TIME KEEPING mode
    reset_tb = 1;
    mode_tb = 0;
    set_tb = 0;
    #10;
    reset_tb = 0;
    #10;
    // go to STOP WATCH mode
    mode_tb = 1; // B=high to go from TIME KEEPING to STOP WATCH mode
    #10; #10; #10; #10; #10; // 5 times to reach STOP WATCH
    // now we need to start counting and then split and then split release and
then stop counting
    mode_tb = 1; // mode=high to START counting
    #10;
    mode_tb = 0; // mode=low to continue in counting
    #100; // wait for some time to let stopwatch count
    set_tb = 1; // A=high to SPLIT
    #10;
    set_tb = 0; // A=low to stay in SPLIT
    #100; // wait for some time in SPLIT mode
    set_tb = 1; // A=high to RELEASE SPLIT
```

```
    #10;
    set_tb = 0; // A=low to continue counting
    #50; // wait for some time to let stopwatch count
    mode_tb = 1; // mode"c"=high to stop counting
    #100;
    $stop;
end


// Design instantiation

SMART_WATCH_TOP DUT (
.CLK(clk_tb),
.reset(reset_tb),
.mode_button(mode_tb),
.set_button(set_tb),
.H1_out(Hour1_tb),
.H0_out(Hour0_tb),
.M1_out(Min1_tb),
.M0_out(Min0_tb),
.Alarm_beep(Alarm_beep_tb)
);

endmodule
```

Chapter 5 Simulation

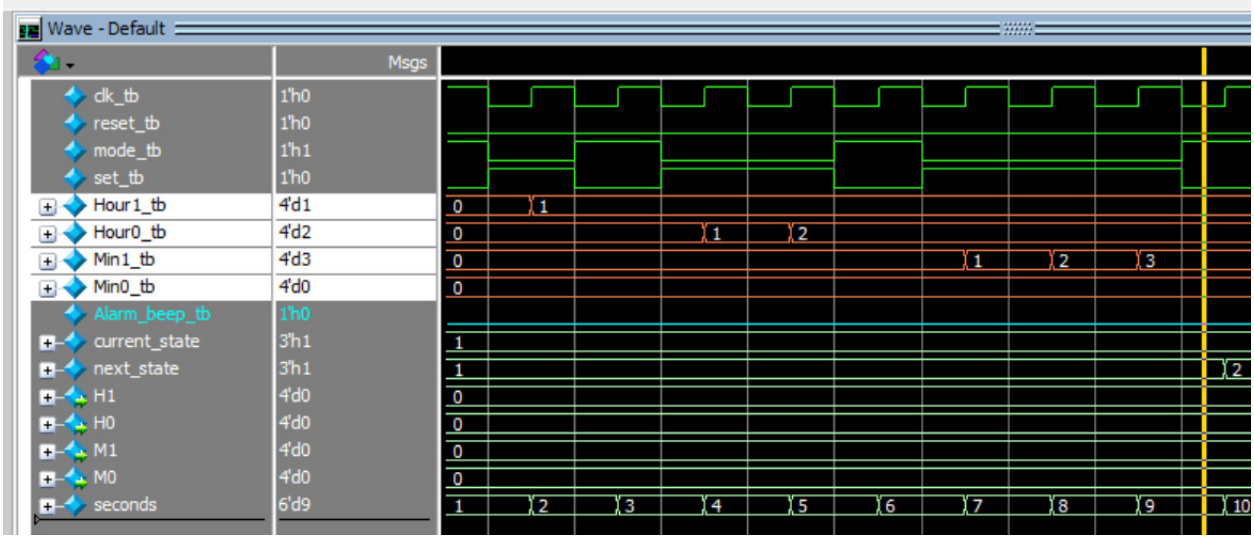
We simulated the TB in clock 1Hz to be one second for each clock period.

1)Reset test screenshot



As we see the CS=00 “TIME KEEPING” mode

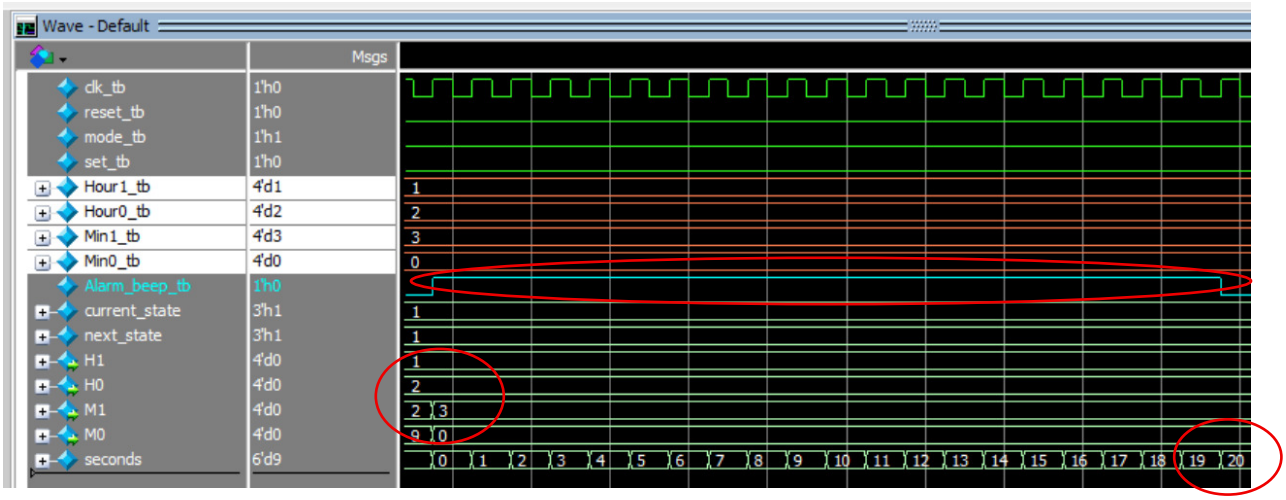
2)SET ALARM to 12:30



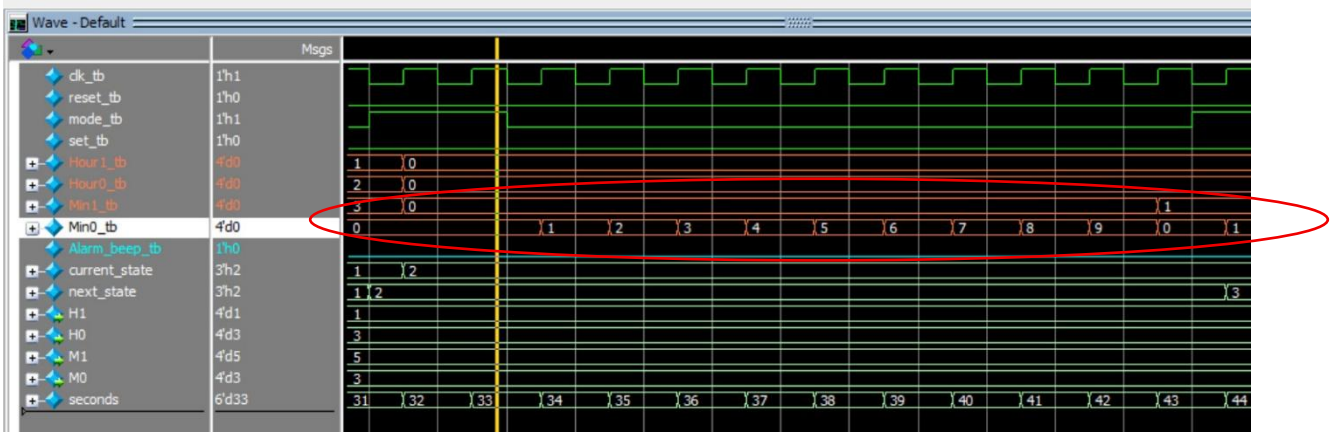
The lower H1,H0,M1,M0,seconds is the normal timing mode that continue counting in background.

As we see we set the ALARM H1_tb , H0_tb , M1_tb, M0_tb to 12:30.

The next waveform shows that the alarmbeeb=1 at system clock 12:30 for 20 seconds.



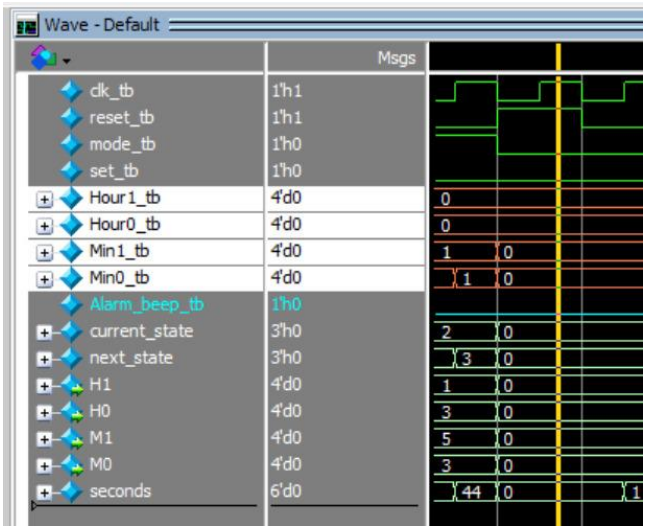
3)test stop watch mode



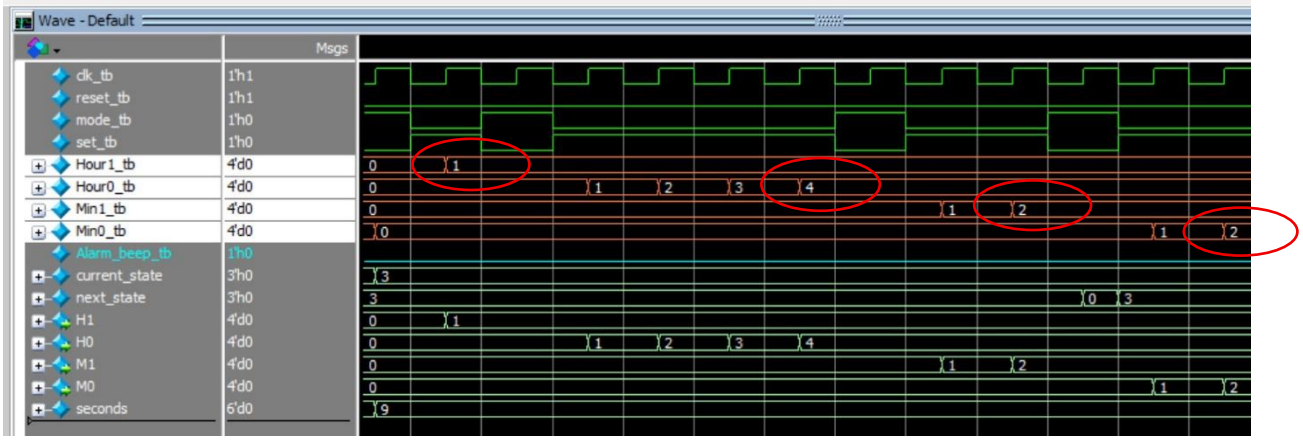
START Stopwatch; stopwatch starts counting from 00:00 where mm:ss till we stop counting when pressing on mode at second 11 , here the display of the watch is H1H0:M1M0 but in stop watch mode it's M1M0:S1S0 so the M1M0 in waveform is S1S0.

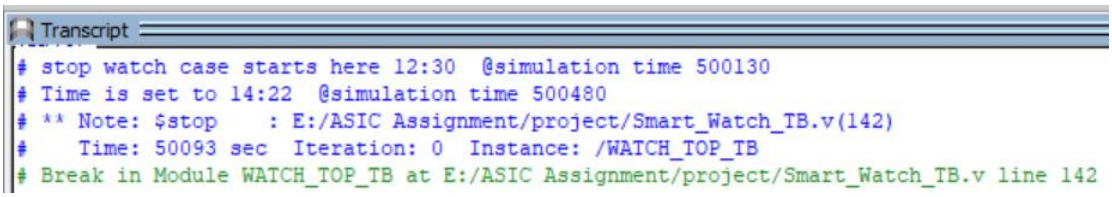
4) test reset again and then TIME SETTING mode to 14:22

Reset so we gone to state 0 "TIME KEEPING"

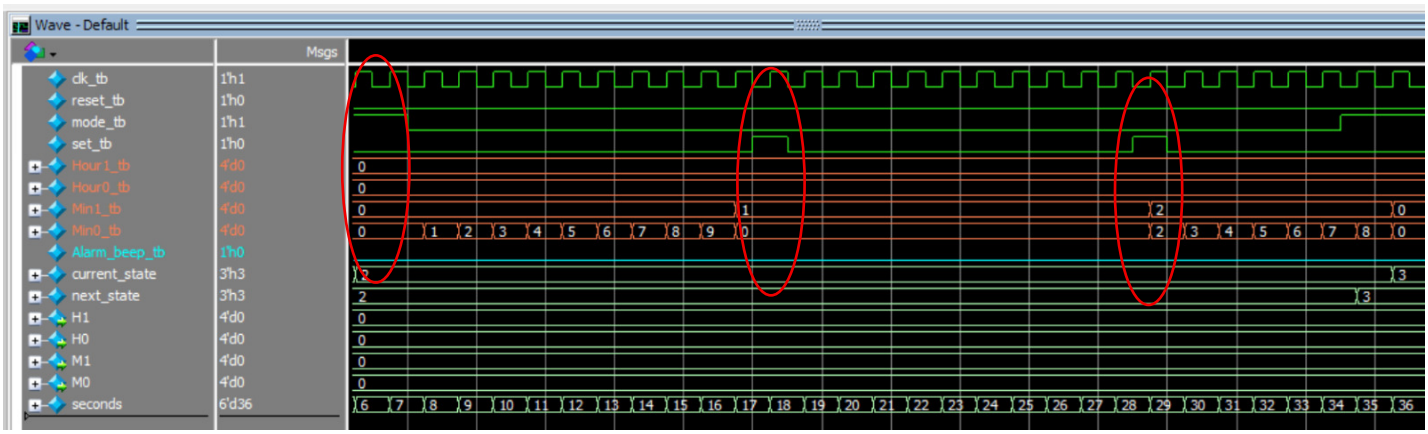


As we see the display is set now to 14:22





5) test split in STOP WATCH



At first red circle the STOP WATCH start counting then 2nd circle we split "set=1" so display stop counting and stop at time=10 from counting in stop watch , then delay 12 cycle and at 3rd circle split release "set=1" so continue counting from time=22 as shown in the wave form.

-The split function pauses the display while still counting in the background, the release lets the display continue with the counter like normal "as we continued counting while pausing the display".